VOLUME III · COMPLETE COVERAGE · ASSIGNMENTS ANSWERED · OUTLINE GAPS

CMS 704 — Everything Left in the Notes and the Outline

The gap-closer. Volumes I and II cover and drill what was taught; this volume answers the three class assignments, reconstructs the two faint classwork minimizations, and supplies the outline topics that never reached the notes at all — floating point, memory technologies, virtual memory, control systems and I/O bus control.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 05 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

1 Coverage map — the notes, the extras and the outline

TOPIC	SOURCE	COVERED IN	NOTE
Architecture · organization	Class p.1–2	Vol I §1–2	Both seven-item lists reproduced in full
Logic gates · truth tables	Class p.2–4	Vol I §3	All seven gates drawn
Boolean algebra laws	Class p.5	Vol I §4	—
Minimization · why · techniques	Class p.5–6	Vol I §5	—
Majority vote circuit	Class p.6	Vol I §5	Fully worked
Classwork 1 and 2 minimizations	Class p.7	**Vol III §3**	**Faint in the scan — method and reconstruction here**
Karnaugh maps	Class p.8	Vol I §6	—
RTL · micro-operations · bus	Class p.8–10	Vol I §7	—
Fetch–execute cycle	Class p.10	**Vol III §4**	**Only sketched in the notes — expanded in RTL here**
Number bases · conversions · arithmetic	Class p.10–14	Vol I §8	—
Signed / unsigned numbers	Extras (textbook §8.7)	Vol I §9	All three schemes worked
Memory hierarchy · cache · locality	Extras p.1–6	Vol I §10	—
Performance · Iron Law	Extras p.7–8	Vol I §11	—
Assignment: number vs digit	Class p.10	**Vol III §2**	**Answered here**
Assignment: truth table for A·B·C and A+B+C	Class p.3	**Vol III §2**	**Answered here**
Assignment: hexadecimal, computer storage and encoding schemes	Class p.14	**Vol III §2**	**Answered here**
Fixed and floating point representation	Outline only	**Vol III §5**	**Never taught — standard material supplied**
Memory technologies and addressing	Outline only	**Vol III §6**	**Magnetic recording, semiconductor memory, magnetic bubbles**
Virtual memory	Outline only	**Vol III §7**	**Never taught**
Control systems: hardware and micro-programmed control	Outline only	**Vol III §8**	**Named on the outline and inside the organization list**
I/O and bus control	Outline only	**Vol III §9**	**Never taught**

Read this before §5–§9. Those five sections are **not** from the lecturer's notes — they are standard textbook material covering topics the **official course outline** lists but the class never reached. They are here because an examiner may set from the outline. Where the notes and this material ever differ, **the notes win**. Learn them after everything in Volumes I and II is secure, not before.

2 The three class assignments, answered

Assignment 1 (page 10) — what is the difference between a number and a digit?

A digit is a symbol; a number is a quantity. A **digit** is one of the individual symbols a number system provides for writing values — base 10 has ten digits (0–9), base 8 has eight (0–7), base 2 has two (0 and 1) and base 16 has sixteen (0–9 and A–F). A **number** is the actual **value or quantity** being represented, which is built by placing digits in positions, each position carrying a weight equal to the base raised to that position.

So the same number can be written with completely different digits depending on the base, while the quantity itself never changes:

$4832_{10} = 11340_{8} = 12E0_{16} = 1001011100000_{2}$ — one number, four sets of digits

BASIS	DIGIT	NUMBER
What it is	A single written **symbol**	A **quantity** or value
How many exist	Exactly as many as the base — base *b* has *b* digits	Infinitely many
Depends on base	**Yes** — the available symbols change with the base	**No** — the quantity is the same in every base
Example	In 4832, the digits are 4, 8, 3 and 2	4832 is the number they jointly represent
Value	Has value only through its **position**	Is the total of all its weighted digits

A neat closing line: *digits are the alphabet of a number system; numbers are the words written with them.*

Assignment 2 (page 3) — truth table for A·B·C and A+B+C

Three inputs, so **$2^3 = 8$ rows**. Fill the input columns by counting up in binary from 000 to 111 so that no combination is missed.

A	B	C	A·B·C	A+B+C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	**1**	1

> **The two sentences that finish the answer.**
> A three-input **AND** outputs 1 in **exactly one** of the eight rows — the last, where every input is 1. A three-input **OR** outputs 0 in **exactly one** row — the first, where every input is 0. That mirror image is the general rule: an *n*-input AND has a single 1 in its column; an *n*-input OR has a single 0.

If the question also asks for the gate symbols, draw the two three-input gates — the same shapes as in Volume I §3, with three input lines instead of two — and give the expressions `A·B·C` and `A+B+C`.

Watch the follow-up: examiners often extend this to `(A·B·C)'` and `(A+B+C)'`, which are simply those two columns inverted, and which De Morgan turns into `A'+B'+C'` and `A'·B'·C'` respectively.

Assignment 3 (page 14) — hexadecimal, computer storage and encoding schemes

WHY HEXADECIMAL AND COMPUTER STORAGE FIT TOGETHER

Computer storage is organised in **bits** grouped into **bytes of 8 bits**. Because $16 = 2^4$, **one hexadecimal digit represents exactly four bits** (a **nibble**), so **two hexadecimal digits represent exactly one byte**. The mapping is exact and needs no arithmetic, which is why memory addresses, memory dumps, machine code, colour values and MAC addresses are all written in hex.

```
1 hex digit  = 4 bits  = 1 nibble
2 hex digits = 8 bits  = 1 byte      e.g. FF = 11111111 = 255
4 hex digits = 16 bits = 1 word

11010011₂  →  1101 0011  →  D3₁₆
```

The point to state: binary is what the hardware stores, but it is long and error-prone for a human to read; hexadecimal is a **compact, lossless shorthand for binary** — four times shorter, with every digit still mapping cleanly onto a fixed group of bits. Octal does the same job in groups of three, but hex matches the 8-bit byte exactly, which is why it dominates.

THE THREE ENCODING SCHEMES

SCHEME	WHAT IT IS
BCD Binary Coded Decimal	Each **decimal digit** is encoded separately in **4 bits**, using only the codes 0000–1001. So `59` is stored as `0101 1001`, **not** as the pure binary `111011`. It wastes six of the sixteen codes but makes decimal display and arithmetic straightforward, which is why it is used in calculators, digital clocks and financial hardware.
EBCDIC Extended Binary Coded Decimal Interchange Code	An **8-bit** character encoding developed by IBM for its mainframes, giving **256** possible characters. It extends the BCD idea to letters, digits and control characters. Largely confined to IBM mainframe systems.
ASCII American Standard Code for Information Interchange	A **7-bit** character encoding giving **128** characters — letters, digits, punctuation and control codes. **Extended ASCII** uses 8 bits for 256. It is the basis of text representation on virtually all modern systems: `'A' = 65₁₀ = 41₁₆`, `'a' = 97₁₀`, `'0' = 48₁₀`.

> **The distinction that earns the mark:** a **number system** (binary, octal, hex) represents **quantities**; an **encoding scheme** (BCD, EBCDIC, ASCII) assigns bit patterns to **symbols and characters**. `0011 0101` is the number 53 in pure binary, the digits "35" in BCD, and the character "5" in ASCII — the same bits, three different meanings, which is exactly why you must always state the scheme in use.

3 The two classwork minimizations — method and reconstruction

The groupings on page 7 of the notes are **faint and partly illegible**, so the exact minterms cannot be recovered with certainty. What **is** certain is the method, which is identical to the majority-vote example: **pair two minterms that differ in exactly one variable, factor that variable out, and let `X + X' = 1` delete it.** Both classworks are reconstructed below from the terms that are legible; treat the method as examinable and re-derive from the truth table on the day.

CLASSWORK 1 — AS RECORDED

```
F = A'BC' + AB'C' + AB'C + ABC
      ①       ②       ③      ④

① and ② ⇒ A'BC' + ABC'  ⇒ BC'(A'+A) = BC'
② and ③ ⇒ AB'C' + AB'C  ⇒ AB'(C'+C) = AB'
③ and ④ ⇒ AB'C  + ABC   ⇒ AC(B'+B)  = AC

∴ F = BC' + AB' + AC
```

Note how each pair is chosen: the two terms are identical except that **one variable appears once plain and once complemented**. That variable is the one that disappears.

CLASSWORK 2 — AS RECORDED

```
X = A'B'C' + A'B'C + AB'C + ABC
      ①        ②       ③      ④

① and ② ⇒ A'B'(C'+C) = A'B'
② and ③ ⇒ B'C(A'+A)  = B'C
③ and ④ ⇒ AC(B'+B)   = AC

∴ X = A'B' + B'C + AC
```

The recorded answer line reads `AB' + A'C + B'C`, which does not follow from these four terms — evidence that some minterms in the scan are misread. **Trust the method, not the copied line.**

THE SAFE PROCEDURE IN THE EXAM, WHATEVER TERMS YOU ARE GIVEN

1. **Build the truth table** even if it is not asked for — it is your only check that no minterm is missing.
2. Write the **sum of products**: one product per row where the output is 1, complementing each input that is 0 in that row.
3. **Pair minterms differing in one variable.** A term may be reused in several pairs — that is legal, because `X + X = X`.
4. Factor and apply `X + X' = 1`, then `Y·1 = Y`.
5. **Verify**: substitute two or three input combinations into your minimized expression and confirm it reproduces the truth table.

4 The fetch–execute cycle in RTL

The notes state the cycle as **Fetch → Decode → Execute** and record that **the PC keeps track of the next instruction to be fetched by the CPU from memory**. Written out in the RTL of Volume I §7, the fetch phase is the standard sequence below — a question asking you to "describe the fetch–execute cycle using register transfer notation" is answered with exactly this.

STEP	RTL	WHAT HAPPENS
T_0	`MAR ← PC`	The address of the next instruction is copied from the program counter into the memory address register
T_1	`MBR ← M[MAR], PC ← PC + 1`	The instruction is read from memory into the memory buffer register while the PC is incremented — both in the same clock cycle, which is what the comma means
T_2	`IR ← MBR`	The instruction is transferred into the instruction register, where the control unit will **decode** it

THE REGISTERS TO NAME

PC — program counter, holds the address of the next instruction · **MAR** — memory address register, holds the address being accessed · **MBR/MDR** — memory buffer (data) register, holds the value read or written · **IR** — instruction register, holds the instruction being decoded · **CU** — control unit, decodes and issues control signals · **ALU** — arithmetic and logic unit, performs the operation.

Why the PC is incremented during T_1 rather than later: the increment uses the ALU and the memory read uses the bus, so the two are independent and can be done in the same cycle. This is a small but real example of the **parallelism mechanisms** in

STEP	RTL	WHAT HAPPENS
T_3...	`execute micro-ops`	The control unit issues the micro-operations that carry out the instruction — arithmetic, logic, transfer or I/O

the organization list — and it is exactly the kind of point that separates a full-mark answer from an average one.

5 Fixed and floating point representation OUTLINE ONLY

The outline names "*fixed and floating point systems, representation*". Neither reached the class notes. This is the standard treatment.

FIXED POINT

A **fixed-point** number reserves a **fixed number of bits for the fractional part**, so the binary point never moves. An 8-bit number with 4 fractional bits stores values in steps of 1/16.

```
0101.1100₂ = 4 + 1 + 0.5 + 0.25 = 5.75
```

- **Advantages** — simple hardware, fast, exact within its step size; ordinary integer circuits can be reused.
- **Disadvantages** — a **narrow range**; very large and very small values cannot both be represented.
- **Used in** embedded systems, digital signal processing and financial arithmetic.

FLOATING POINT

A **floating-point** number stores a number in the form `± mantissa × base^exponent`, so the binary point **floats** — a far wider range for the same number of bits, at the cost of precision.

```
value = (−1)^S × 1.M × 2^(E − bias)
```

IEEE 754	SIGN	EXPONENT	MANTISSA	BIAS
Single (32-bit)	1	8	23	127
Double (64-bit)	1	11	52	1023

Normalisation: the mantissa is shifted so there is exactly one non-zero digit before the point; in binary that digit is always 1, so it is not stored — the "hidden bit", which buys one extra bit of precision free.

BASIS	FIXED POINT	FLOATING POINT
Binary point	Fixed in one place	Moves — encoded in the exponent
Range	**Narrow**	**Very wide**
Precision	Uniform across the range	Relative — large values lose absolute precision
Hardware	Simple, reuses integer circuits	Needs a dedicated floating-point unit (FPU)
Speed	Faster	Slower, and measured in **FLOPS**
Error	Quantisation error only	**Rounding error accumulates**; 0.1 has no exact binary form

6 Memory technologies and addressing OUTLINE ONLY

The outline names "*general characteristics of memory operation (technology — magnetic recording, semiconductor memory, complex devices, magnetic bubbles), memory addressing*".

SEMICONDUCTOR MEMORY

Built from transistors on silicon; the technology of registers, cache and main memory.

- **SRAM** — static RAM; holds its value while powered, ~6 transistors per bit, **fast and expensive**; used for **cache**.
- **DRAM** — dynamic RAM; one transistor and a capacitor per bit, must be **refreshed** thousands of times a second, **slower but dense and cheap**; used for **main memory**.
- **ROM family** — ROM, PROM, EPROM, EEPROM and **flash**; non-volatile, used for firmware and SSDs.

MAGNETIC RECORDING

Data stored as the **direction of magnetisation** of tiny regions on a coated surface, read and written by a head moving over it.

- **Hard disk** — rotating platters, random access in milliseconds, terabyte capacities.
- **Magnetic tape** — **sequential access** only, very cheap per bit, used for archive and backup — the bottom level of the hierarchy.
- **Non-volatile**, but mechanical, so far slower than anything electronic.

MAGNETIC BUBBLE MEMORY

A **non-volatile** technology in which tiny magnetised regions — "**bubbles**" — are moved through a thin magnetic film by an external field; the presence or absence of a bubble at a position is one bit.

- No moving parts, so it was rugged and survived power loss.
- **Obsolete** — overtaken in the 1980s by cheap semiconductor and flash memory.
- Named on the outline as an example of a "**complex device**"; know what it is, not how to design one.

MEMORY ADDRESSING

MODEL	HOW IT WORKS
Byte addressable	Every **byte** has its own address — the model used by almost all modern machines. Larger units are addressed by the address of their first byte.
Word addressable	Each address refers to a whole **word** (e.g. 32 bits). Fewer addresses are needed but individual bytes cannot be reached directly.

`n address lines → 2ⁿ addressable locations`

16 address lines address **64 K** locations; 32 lines address **4 G**. This is the same 2^n counting rule as truth-table rows and unsigned ranges — one idea, three appearances on this syllabus.

VOLATILE VS NON-VOLATILE — A LIKELY ONE-LINER

VOLATILE	NON-VOLATILE
Registers, cache (SRAM), main memory (DRAM)	ROM/flash, SSD, hard disk, optical disk, magnetic tape, magnetic bubble
Contents lost when power is removed	Contents retained when powered down

The link back to the hierarchy: the top three levels are volatile and electronic; the bottom levels are non-volatile and (historically) mechanical. That is exactly why the pyramid's lower levels are so much slower — and why a computer must load programs upward from disk into RAM before it can run them.

7 Virtual memory OUTLINE ONLY

Virtual memory is a memory-management technique in which the operating system gives each program the illusion of a **large, contiguous private address space**, while the actual data is spread across physical RAM and secondary storage. It extends the memory-hierarchy idea downward: **cache makes RAM look faster; virtual memory makes disk look like RAM.**

HOW IT WORKS

1. The virtual address space is divided into fixed-size **pages**; physical memory is divided into **frames** of the same size.
2. A **page table** maps each virtual page to the physical frame holding it.
3. The **MMU** (memory management unit) performs this translation on every access, in hardware.
4. If the page is not in RAM, a **page fault** occurs: the OS fetches the page from disk, evicts another page if necessary, updates the page table and restarts the instruction.
5. The **TLB** (translation lookaside buffer) caches recent translations so the page table need not be consulted every time — the TLB is named in the **organization** list in Volume I §2.

WHY IT IS WORTH HAVING

- **Programs larger than physical RAM can run** — only the active pages need be resident.
- **Protection and isolation** — one process cannot address another's memory, because the page tables differ.
- **Simpler programming** — every program sees the same flat address space, whatever the machine has installed.
- **Efficient sharing** — two processes can map the same physical frame, e.g. for shared libraries.

> **The cost — thrashing.** If the working set of active pages exceeds physical memory, the system spends more time swapping pages than executing instructions and performance collapses. Note the connection to Volume I §10: **virtual memory works for the same reason cache does — locality of reference.** A page fault is to RAM what a cache miss is to cache, only thousands of times more expensive.

8 Control systems: hardware vs micro-programmed control OUTLINE ONLY

The outline names "*control systems, hardware control, micro-programmed control, asynchronous control*", and "**hard-wired control vs micro-programmed control**" is item 2 of the organization list in Volume I §2 — so this is the most examinable of the outline gaps.

The **control unit** is the part of the CPU that **decodes each instruction and generates the sequence of control signals** that drive the data path — telling registers when to load, the ALU which operation to perform and memory when to read or write. There are two ways to build it.

BASIS	HARD-WIRED (HARDWARE) CONTROL	MICRO-PROGRAMMED CONTROL
Implementation	A **fixed logic circuit** — gates, decoders, counters and flip-flops — designed for one instruction set	A **control memory** holding **microinstructions**; each machine instruction runs a small microprogram
Speed	**Faster** — signals come straight out of combinational logic	**Slower** — each step requires a control-memory read
Flexibility	**Rigid** — changing the instruction set means redesigning the circuit	**Flexible** — change the instruction set by rewriting the microcode
Complexity	Hard to design and debug for a large instruction set	Systematic and easier to design, test and extend
Cost	Cheaper in silicon for a **small, simple** instruction set	Extra control memory, but cheaper for a **large, complex** one
Typically used in	**RISC** processors, with few, uniform instructions	**CISC** processors, with many complex, variable-length instructions

SYNCHRONOUS CONTROL

All operations are timed by a **common clock signal**; every micro-operation completes within a fixed clock period, and the clock cycle must be long enough for the **slowest** unit. Simple to design and to reason about — this is what the RTL notation in Volume I §7 assumes, with the comma meaning "in the same clock cycle".

ASYNCHRONOUS CONTROL

There is **no common clock**. Units coordinate through **handshaking** signals — typically a **request** from the sender and an **acknowledge** from the receiver — so each operation takes exactly as long as it needs. **Advantages:** average-case rather than worst-case speed, lower power, no clock-distribution problem. **Disadvantages:** harder to design and verify, and more control hardware. It is the standard method for **I/O transfers**, where device speeds vary wildly.

9 I/O and bus control OUTLINE ONLY

THE SYSTEM BUS — THREE GROUPS OF LINES

BUS	CARRIES
Address bus	The address of the location or device being accessed. **Unidirectional**; its width fixes how much memory is addressable (2^n).
Data bus	The data itself. **Bidirectional**; its width is a major factor in throughput.
Control bus	The control and timing signals — read, write, interrupt request, bus grant, clock.

This is the hardware behind the **bus notation** in the notes: `Bus ← R1` means R1 drives the data bus, and `R2 ← Bus` means R2 latches from it. Only one device may drive the bus at a time, which is why **bus arbitration** exists — a scheme deciding which unit becomes bus master when several request it at once.

THE TWO I/O ADDRESSING MODELS

MODEL	HOW DEVICES ARE REACHED
Memory-mapped I/O	Device registers occupy addresses in the **ordinary memory address space**, so normal load and store instructions work on them. Simpler instruction set, but it consumes memory addresses.
Isolated (special) I/O	Devices live in a **separate I/O address space** reached by dedicated IN and OUT instructions. Memory space is preserved, at the cost of extra instructions and control lines.

These are the two options listed as item 7 of the **architecture** list — "*input/output model: memory-mapped I/O or special I/O instructions*".

THE THREE TECHNIQUES FOR TRANSFERRING DATA TO AND FROM A DEVICE

TECHNIQUE	HOW IT WORKS	COST / BENEFIT
Programmed I/O (polling)	The CPU repeatedly **checks the device's status** until it is ready, then performs the transfer itself.	Simplest, but **wastes CPU time** in the polling loop.
Interrupt-driven I/O	The device raises an **interrupt** when ready; the CPU suspends its work, saves state, runs the interrupt service routine and resumes.	The CPU does useful work while waiting — but every word still passes through it.
DMA (direct memory access)	A **DMA controller** transfers a whole block **directly between the device and memory**, interrupting the CPU only when the block is complete.	**Fastest** for bulk transfers; the CPU is bypassed entirely during the transfer.

Note the link to the **architecture** list item 6 — *interrupt and exception handling: how the processor responds to external events or internal faults*. An **interrupt** is an external event, e.g. a device becoming ready; an **exception** (or trap) is an internal fault, e.g. division by zero or a **page fault** (§7).

10 One-page note map — the whole course in a grid

TOPIC	THE SINGLE LINE YOU MUST BE ABLE TO WRITE	THE NUMBER TO QUOTE
Architecture	The ISA — the rules defining functionality, organization and implementation as seen by the programmer; the hardware–software contract	7 things it specifies

TOPIC	THE SINGLE LINE YOU MUST BE ABLE TO WRITE	THE NUMBER TO QUOTE
Organization	The actual implementation — how functional units are constructed, interconnected and controlled	7 things it specifies
Logic gate	A device performing a boolean function on binary inputs, giving one binary output	7 gates · rows = 2^n
Boolean algebra	Identity, null, idempotent, complement, absorption, distributive, De Morgan	A + A' = 1 drives minimization
Minimization	Simplifying boolean equations to reduce the hardware required	3 techniques · 3 reasons
K-map	A graphical matrix method; group the 1s in powers of 2 and cancel the changing variable	Columns 00 01 11 10
Majority vote	A circuit whose output follows the majority of its inputs	F = AB + BC + AC
RTL	Symbolic notation describing micro-operations between hardware registers	5 symbols · 4 micro-op types
Fetch–execute	MAR ← PC; MBR ← M[MAR], PC ← PC+1; IR ← MBR; execute	Fetch → decode → execute
Number bases	Divide up for whole numbers, multiply down for fractions, expand positionally to return	3:1 octal · 4:1 hex
Unsigned	No negative values; n bits give 2^n codes	Range 0 … $2^n - 1$
Signed	The MSB is the sign bit — 1 negative, 0 positive	SM · 1's · 2's = 1's + 1
Encoding schemes	Assign bit patterns to symbols, not quantities	BCD 4-bit · EBCDIC 8-bit · ASCII 7-bit
Fixed / floating point	Fixed binary point vs mantissa × base ^ exponent	IEEE 754: 1 + 8 + 23 bits, bias 127
Memory hierarchy	A pyramid categorizing storage by speed, cost and capacity	5 levels, registers → tape
Cache	High-speed volatile memory between the CPU and RAM holding frequently used data	L1 2–64 KB · L2 256–512 KB · L3 1–8 MB
Cache mapping / writing	Direct, fully associative, set associative; write-through vs write-back	3 mappings · 2 policies
Locality	A program's tendency to reuse the same or nearby locations over a short period	Temporal (loop) · spatial (array)
Virtual memory	The illusion of a large contiguous private address space, paged between RAM and disk	Page · frame · page table · TLB · page fault
Control unit	Decodes instructions and generates the control signals driving the data path	Hard-wired (RISC) vs micro-programmed (CISC)
Buses and I/O	Address, data and control lines; memory-mapped or isolated I/O	Programmed · interrupt-driven · DMA
Performance	How quickly and efficiently a system executes a workload; the reciprocal of execution time	CPU Time = IC × CPI × cycle time